MEMORY-EFFICIENT DEEP HEDGING

NATANAEL ALPAY AND EMERIC BATTAGLIA

Abstract. We study how to train neural hedging strategies with reduced memory complexity without incurring exorbitant runtime increases. In a three-stock experiment, generating market scenarios during training reduced peak process RAM by 63.8%, with nearly the same test CVaR risk. Additionally, we also gain 2.47× speedup through parallel simulation. The project combines an modified implementation of Deep Hedging with experiments that measure the cost of storing, generating, and training on synthetically generated underlyings following a Heston model.

1. Introduction

An option seller uses hedging trades to offset uncertain future payments. A Deep hedging network seeks to learn these hedges from market scenarios while accounting for transaction costs. Following Bühler et al. [1], we implement this approach in a Heston market, where volatility changes over time. At each trading date, a neural network uses stock prices, variances, and previous holdings to choose the next holdings. The daily networks are trained together to reduce hedge's final the risk/loss.

Our main change concerns the generation of training data. The original Deep hedging network uses Fixed-pool training, which repeatedly samples portfolio outcomes from a stored dataset. We implement Dynamic training, which generates a batch of portfolio outcomes each iteration and releases it after use. We compare the two network implementations with the same network architecture, batch normalization, initial weights, and update budget. In the project, we use two forms of risk: the average of the worst half of losses, i.e. CVaR 50%, and entropic risk, which penalizes large losses exponentially. Validation at regular intervals selects the policies, which we then evaluate on a shared test set.

2. Memory savings and hedging results

The main experiment hedges three calls on independent stocks over 30 trading dates, using the stocks and three variance claims. This particular portfolio serves as a proof-of-concept, and extensions to true large-scale portfolios are discussed later. Both configurations have 64,980 parameters, face 0.1% transaction costs, and use 8,192 test scenarios. The fixed-pool network has a train size of 131,072 stored scenarios. The fixed pool and dynamic-training networks use batches of size 256. Each configuration receives 100 warm-up updates followed by 500 updates per risk objective.

Department of Mathematics, University of California, Irvine. Email: nalpay@uci.edu (N. Alpay); ebattagl@uci.edu (E. Battaglia).

Measurement	Fixed pool	Dynamic
Peak process RAM (MiB)	1,399.1	506.0
Test CVaR	9.095	9.031
Test entropic risk	29.533	20.859
Preparation + training + testing (s)	97.0	170.9

Figure 1. Peak RAM, risks, and training times are reported for each network.

Dynamic-training used 36.2% of the baseline fixed-pool peak RAM, with a 0.7% lower observed CVaR and 29.4% lower entropic risk. The network itself was unchanged. The trade-off was 76.2% longer elapsed time, which led us to investigate faster portfolio outcome generation.

3. Faster generation and next steps

We propose a 'Fast generator' that splits independent the task of generating batches of portfolio paths across six (or more) CPU workers. To implement this in dynamic training, a queue holds up to two upcoming batches, allowing the simulation process to overlap with training.

In a separate short benchmark with three training seeds, the fast generation option reduced the median dynamic preparation, training, and testing time from 15.27 to 6.18 seconds, a 2.47× speedup. This gain does require additional CPU capacity. The benchmark uses 10 warm-up updates and 25 continuation updates per objective, so it does not establish the same speedup for the longer run above.

4. Conclusion

Our contribution is a working, reproducible hedging framework and a direct comparison of memory, runtime, and test risk between ours and the original Deep Hedging network. The memory comparison uses one training seed, so repeated runs are needed to assess how consistently the risk results hold. Next we plan to test portfolios with a significantly larger number of stocks and trading dates. The faster generation scheme and the reduced memory complexity gives rise to a promising model that will make significantly larger hedging problems tractable even with limit compute and memory resources.

References. [1] H. Bühler, L. Gonon, J. Teichmann, and B. Wood, *Deep Hedging*, arXiv:1802.03042 (2018). [2] Project repository, snapshot `bc0b968`: saved outputs in the small-portfolio and path-generation notebooks and the portfolio memory reports.